One-Day Revision Notes: Stock Order Matching Engine (C++)

Repository: shivamkachhadiya/Stock_Order_Matching_Engine_Cpp

Source-driven summary based on: include/order_book.hpp, src/order_book.cpp, include/matching_engine.hpp, src/matching_engine.cpp, include/price_level.hpp, include/lock_free_queue.hpp, src/stress_test.cpp.

1) Quick Architecture Snapshot

• **Core entities**: Order(id, side, price, qty, timestamp), Trade(buyId, sellId, price, qty).

• **Order book state**: bids and asks are std::map<int, PriceLevel> (price → FIFO queue at that price).

• **Price level**: std::deque<Order> for strict time priority within same price.

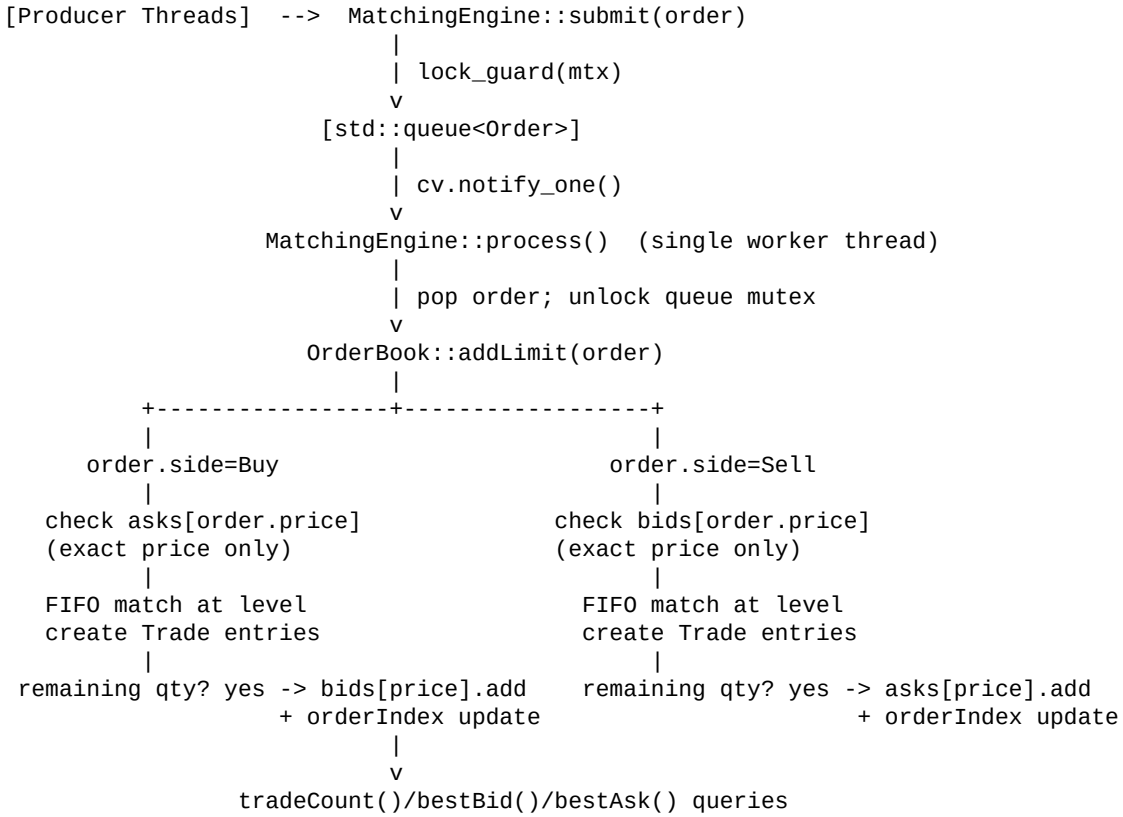
• **Order index**: unordered_map<orderId, (side, price)> used for cancellation lookup.

• **Execution log**: vector<Trade> stores all generated trades.

• **Concurrency model**: multi-producer threads submit to one shared queue; one worker thread performs deterministic matching.

• **Current queue implementation used**: std::queue + mutex + condition_variable. A lock-free ring buffer implementation exists but is commented out in MatchingEngine.

2) Order-Matching Lifecycle Diagram (Readable in PDF)



3) Detailed Revision Notes (One-Day Focus)

A. Data structures and why they matter

• std::map for bid/ask levels gives ordered keys, making best bid = rbegin() and best ask = begin().

• PriceLevel uses deque so front() and pop_front() are O(1), naturally implementing FIFO per price.

• orderIndex enables O(1) average lookup by orderId for cancellation routing to side + price.

B. Matching logic in current implementation
• Limit buy checks asks.find(order.price) and matches only at that exact price level.
• Limit sell checks bids.find(order.price) and matches only at that exact price level.
• Market orders repeatedly consume best opposite prices until qty is exhausted or book side is empty.
• Trades are appended as Trade{buyId, sellId, price, qty}.

C. Cancellation flow
• cancel(orderId) uses orderIndex to find side/price.
• It reconstructs that price-level queue excluding the target order (deque copy-through temp).
• orderIndex entry is removed after successful cancel.

D. Complexity (as implemented)
• bestBid/bestAsk: O(1).
• add to known price level: amortized O(1) for deque push + O(log P) map access.
• cancel: O(K) for K orders at that price level (full level scan/rebuild).
• market order: proportional to consumed resting orders/levels.

E. Concurrency/performance considerations seen in code
• Deterministic matching is protected by single worker thread (no book races).
• Ingestion path currently takes a mutex (queue push), so contention can rise with many producers.
• LockFreeQueue exists and could reduce ingestion lock contention if integrated correctly.
• stress_test.cpp uses 10 producer threads × 100k orders each and reports throughput/fill-rate/trades.

F. Important implementation caveats to remember for interviews
• Crossing limit logic is simplified: matching is exact-price only for limit orders, not best-price sweep across multiple levels.
• Filled resting orders are popped from price-level queues, but orderIndex cleanup for naturally filled orders is not explicit.
• Empty price levels remain in maps; print and quantity paths guard with level.empty().
These are excellent discussion points when asked about production-hardening.

4) FAANG-Style Interview Questions with Strong Answers

1. Why use a single matching thread? It preserves strict deterministic sequencing and avoids complex locking on the order book. This is a common exchange design: parallelize ingress, serialize matching.

2. Why map for bids/asks? Ordered maps let you query best prices quickly (begin/rbegin) while keeping all active levels sorted, which simplifies market-order sweeps.

3. How is price-time priority implemented? Price priority comes from map ordering; time priority comes from deque FIFO per price level.

4. What is cancellation complexity and why? Average O(1) to locate via orderIndex, but O(K) to rebuild that level because the implementation scans all orders at that price.

5. How would you make cancellation O(1)? Store intrusive iterators/handles to node positions (linked-list per level + hash map of id→iterator) so delete is direct.

6. Is this implementation truly lock-free end-to-end? No. The ingestion path currently uses std::queue with mutex+CV. A lock-free queue class exists but is not active in MatchingEngine.

7. What does deterministic execution buy you? Replayability, easier debugging/compliance audit, and consistent outcomes under concurrency.

8. How do market orders execute here? They repeatedly hit best opposite price levels until qty is filled or no liquidity remains.

9. What is a key correctness gap in limit matching? Limit orders match only exact opposite price level, not all marketable levels (e.g., buy at 101 should consume ask 100 first).

10. How do you reason about throughput bottlenecks? Measure producer lock contention, queue wake-up overhead, cache misses in maps/deques, and worker saturation. The queue mutex is an immediate hotspot candidate.

11. What data-structure tradeoff exists with std::map? Great ordering semantics but pointer-heavy tree nodes hurt cache locality versus array/tree hybrids in ultra-low latency systems.

12. How would you support multiple symbols? Shard by symbol: one book + one matching thread per symbol partition (or symbol hash group), preserving in-symbol ordering.

13. How should timestamps be used here? They are carried in Order but not used for matching tie-breaks directly because FIFO queue insertion already captures arrival order in this process model.

14. What reliability features are missing for production? Persistence/journaling, recovery snapshots, idempotent replay, risk checks, and robust input validation.

15. How would you test this engine deeply? Unit-test matching edge cases, deterministic replay tests, concurrency stress with invariants, and property-based checks (no negative qty, conservation of volume).

16. Why might lock-free queue integration be non-trivial? You must handle backpressure/full-queue behavior, memory ordering correctness, and shutdown semantics without losing orders.

17. What DSA concepts are central here? Ordered maps (balanced trees), hash maps, FIFO queues/deques, producer-consumer concurrency, and amortized vs worst-case complexity reasoning.

5) Counter / Follow-Up Questions (Concise Answers)

1) What happens if two producers submit same orderId? orderIndex overwrite risk; no explicit duplicate-id guard in current code.

2) Why not process in producer thread directly? Would require heavy locking around the book and break deterministic single-sequence execution.

3) Can bestBid()/bestAsk() race with matching thread? Potentially yes if queried concurrently without external synchronization; currently they read shared state directly.

4) Why is fill rate in stress test an approximate metric? It uses tradeCount/totalOrders, but one trade event can represent partial fills and not full order completion semantics.

5) Does cancel remove empty price levels? No explicit erase of empty map levels after cancel/match.

6) Is market order book impact bounded? By available opposite-side liquidity; loop exits when best price unavailable.

7) Main reason to keep orderIndex? Fast route from orderId to side/price for cancellation path.

8) Where is FIFO guaranteed? Within each PriceLevel deque via push_back + front/pop_front.

9) Why include lock_free_queue.hpp if unused? Likely planned optimization path; currently disabled in MatchingEngine.

10) One immediate production upgrade? Implement true marketable limit crossing (consume best opposite levels up to limit price).

Final revision tip: Practice explaining this engine in three layers—(1) data structures, (2) matching correctness, (3) concurrency/performance bottlenecks and production upgrades.